Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №35

ТЕМА: Рекурсивні функції та алгоритми.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з поняттям рекурсії та рекурсивної функції;
- вивчити структуру рекурсивної функції: базовий випадок і рекурсивний перехід;
- навчитися реалізовувати класичні рекурсивні алгоритми (факторіал, числа Фібоначчі);
- зрозуміти співвідношення між рекурсивним і циклічним розв'язанням однієї й тієї самої задачі.

Теоретичні відомості

1. Що таке рекурсія

Рекурсія — це спосіб розв'язання задачі, при якому функція викликає саму себе для розв'язання «меншої» подібної підзадачі. Кожна коректно побудована рекурсивна функція обов'язково складається з двох частин:

- базовий випадок — найпростіший варіант задачі, для якого відповідь відома одразу, без подальших викликів (саме він зупиняє рекурсію);
- рекурсивний перехід — зведення поточної задачі до такої самої, але простішої (меншої) задачі, з викликом тієї самої функції.

Якщо забути про базовий випадок або сформулювати його неправильно, рекурсія ніколи не зупиниться і призведе до помилки переповнення стека викликів (RecursionError).

2. Приклад: факторіал

Факторіал числа n можна визначити рекурсивно: 0! = 1 (базовий випадок), а для n > 0: n! = n * (n-1)! (рекурсивний перехід):

```
def factorial(n):
    if n == 0:
        return 1          # базовий випадок
    return n * factorial(n - 1)   # рекурсивний перехід

print(factorial(4))
```

Результат: *24*



Рис. 1. Послідовність вкладених викликів factorial(4): кожен виклик чекає результат наступного, і лише після досягнення базового випадку обчислення «розгортаються» у зворотному порядку

Виконання рекурсії відбувається у два етапи: спочатку «занурення» — послідовні виклики factorial(4), factorial(3), factorial(2), factorial(1), доки не буде досягнуто базового випадку; потім «розгортання» — кожен виклик отримує результат від наступного і повертає власний результат тому, хто його викликав (1, потім 2*1=2, потім 3*2=6, потім 4*6=24).

3. Приклад: числа Фібоначчі

Послідовність Фібоначчі (0, 1, 1, 2, 3, 5, 8, 13, ...) визначається рекурсивно: перші два числа дорівнюють 0 і 1 (базові випадки), а кожне наступне — сума двох попередніх:

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(8):
    print(fibonacci(i), end=' ')
```

Результат: *0 1 1 2 3 5 8 13*

Зверніть увагу: тут функція викликає саму себе двічі в одному рекурсивному переході — це типово для задач, де поточний результат залежить не від одного, а від кількох попередніх.

4. Рекурсія проти циклу

Практично будь-яку рекурсивну функцію можна переписати за допомогою циклу, і навпаки. Рекурсивний варіант часто виглядає коротшим і природнішим для задач, які самі по собі мають рекурсивну природу (наприклад, обхід дерева каталогів), тоді як цикл зазвичай ефективніший за витратами пам'яті, оскільки не створює додаткових вкладених викликів.

```
def factorial_loop(n):
    result = 1
    for i in range(1, n + 1):
        result = result * i
    return result

print(factorial_loop(4))
```

Результат: *24*

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані завдань 1–7* залежать від N:

Завдання	**Дані варіанта**	**Приклад N = 7**
1, 2, 4, 5	число N	7
3	n = N + 3	10
7*	пара (6N, 4N)	(42, 28)

б) Кожне завдання — окремий файл Прізвище_ПР35_зN.py (наприклад, Ivanenko_PR35_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) У рекурсивних функціях не використовуйте цикли та готові math.factorial(), math.gcd(), pow() і оператор **. Кожна рекурсивна функція має чітко виписаний базовий випадок. Для великих n перевіряйте обмеження глибини рекурсії.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР35_з1.py. Реалізуйте рекурсивний factorial(n) (як у прикладі 2) і виведіть значення для 0, 5 та N.

   Приклад (N = 7): factorial(0) → 1; factorial(5) → 120; factorial(7) → 5040.

2. Реалізуйте рекурсивну fib(n) (fib(0) = 0, fib(1) = 1), виведіть перші 10 чисел і значення fib(N).

   Приклад (N = 7): 0, 1, 1, 2, 3, 5, 8, 13, 21, 34; fib(7) → 13.

3. Напишіть рекурсивну sum_to_n(n) — суму чисел від 1 до n (базовий випадок sum_to_n(1) = 1) — і виведіть її для n = N + 3.

   Приклад (N = 7): sum_to_n(10) → 55; sum_to_n(1) → 1.

4. Напишіть рекурсивну power(base, exponent) (exponent ≥ 0) без оператора ** і виведіть power(2, 10), power(N, 0), power(N, 3).

   Приклад (N = 7): 1024; 1; 343.

5. Напишіть циклічну версію factorial і виведіть для n = N обидва результати та «Результати збігаються» або «Результати відрізняються». Додайте коментар: яка версія, на вашу думку, зрозуміліша й чому.

   Приклад (N = 7): «Рекурсія: 5040», «Цикл: 5040», «Результати збігаються».

6. Напишіть рекурсивну reverse_string(s), що повертає рядок у зворотному порядку (базовий випадок — довжина 0 або 1) без зрізу [::-1]. Перевірте на своєму прізвищі.

   Приклад: «Python» → «nohtyP»; «» → «»; «a» → «a».

7. Завдання підвищеної складності*. Реалізуйте рекурсивний алгоритм Евкліда gcd(a, b) і виведіть НСД для пари (6N, 4N) вашого варіанта та ще двох пар.

   Приклад (N = 7): gcd(42, 28) → 14; gcd(48, 18) → 6; gcd(17, 5) → 1; gcd(5, 0) → 5.

Критерії оцінювання

Складова	Оцінювання	Макс. балів

Завдання 1–6	кожне по 1,5 бала	9
Завдання 7*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, відсутній базовий випадок, використано цикл чи готову функцію бібліотеки, не збігаються результати двох версій) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити базовий випадок чи вхідні дані). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Що таке рекурсія? З яких двох обов'язкових частин складається рекурсивна функція?
2. Що станеться, якщо в рекурсивній функції неправильно визначити (або забути) базовий випадок?
3. Опишіть своїми словами, як відбувається обчислення factorial(4) — етапи «занурення» і «розгортання».
4. Чому функція обчислення чисел Фібоначчі викликає саму себе двічі в одному рекурсивному переході?
5. У чому основна практична різниця між рекурсивним і циклічним розв'язанням однієї задачі?